Laboratorul 8, Lucrarea B. Unealta stricată, reparată cu git

Arsenalul sub lupă

fișă de laborator · 90 min · se lucrează individual, la terminal

> **CE FACEȚI ASTĂZI.** În depozitul de exercițiu creat la Lucrarea A, exercitiu-git, reparați o unealtă stricată, inventar.py, eroare cu eroare, cu câte un commit după fiecare reparație. Istoria din git devine jurnalul depanării. Dacă nu ați terminat Lucrarea A, terminați-o întâi: aici aveți nevoie de un depozit care face push.

LUCRAREA B (extindere). Unealta stricată, reparată cu git

Obiectiv: aduci inventar.py la starea în care rulează și dă exact ieșirea așteptată. Nu o rescrii: o repari câte o greșeală pe rând, citind fiecare eroare, și faci câte un commit după FIECARE reparație. La final, git log --oneline este jurnalul depanării tale.

B1. Pregătire, în depozitul exercitiu-git (8 min)

Descarcă inventar.py și tinte.json din secțiunea sesiunii de pe ELSE și pune-le într-un folder nou, depanare/, în exercitiu-git. Apoi salvezi starea de pornire, ca să se vadă de unde ai plecat:

```
cd ~/exercitiu-git
git pull
mkdir depanare
# copiaza aici: depanare/inventar.py si depanare/tinte.json
git add depanare
git commit -m "Pornire: inventar.py stricat"
cd depanare
python inventar.py tinte.json --prag 1000
```

Va crăpa imediat. Înainte să atingi codul, uită-te la ce trebuie să iasă la final:

```
Tinte citite: 6
sar peste 192.168.56.103 : port invalid abc
ssh: [22]
http: [80, 8080]
mysql: [3306]
necunoscut: [5432]
Peste 1000: ['192.168.56.102', '192.168.56.103', '192.168.56.104']
Raport salvat in raport_inventar.json
```

B2. Metoda, la fiecare eroare (5 min)

```
1. Citesti ULTIMA linie a traceback-ului: tipul erorii si mesajul.
2. Mergi la linia indicata (File "...", line N).
3. Formulezi o banuiala intr-o propozitie ("portul e text").
4. Schimbi UN singur lucru si rulezi din nou.
5. Cand eroarea a disparut: un commit care spune ce ai reparat.
```

Mesajul de commit este rândul din jurnal. Scrie eroarea și reparația, de exemplu:

```
git commit -am "NameError tinta: numele corect e tinte"
git commit -am "TypeError la prag: type=int in argparse"
```

De observat. `-am` face add și commit într-un singur pas, pentru fișierele pe care git le urmărește deja. Merge aici fiindcă inventar.py e deja în depozit.

B3. Greșelile care nu lasă programul să pornească (12 min)

Primele erori apar înainte să ruleze vreo linie: Python citește tot fișierul și refuză să-l pornească dacă forma nu e corectă. SyntaxError și IndentationError arată linia unde Python s-a împiedicat; uită-te și la linia de deasupra. După „:" urmează mereu un rând indentat.

B4. Greșelile de nume, de tip și de date (30 min)

Odată pornit, programul cade pe rând în erori de rulare, în ordinea în care rulează codul.

Dacă ultima linie spune...	Gândește-te la...
`NameError: name ... is not defined`	un nume scris diferit față de locul unde a fost creat
`TypeError: can only concatenate str`	text lipit cu un număr: str() sau f-string
`KeyError: ...`	o cheie care lipsește la unele ținte: .get(cheie, rezerva)
`'NoneType' object has no attribute`	o funcție care nu întoarce nimic (lipsește return)
`'>' not supported between ... 'str'`	o valoare din terminal rămasă text: type=int
`ValueError: invalid literal for int()`	un port care nu e număr: există deja o funcție care îl tratează

B5. Greșelile tăcute (15 min)

Când unealta nu mai crapă, compară ieșirea, rând cu rând, cu ieșirea așteptată. Mai sunt două greșeli care nu dau nicio eroare: un rezultat gol și o listă căreia îi lipsește un element. Pune un print() temporar în bucla suspectă și uită-te atent la indentarea lui return și la ce se întâmplă cu o listă la fiecare trecere. Și ele primesc câte un commit.

B6. Verificarea finală și push (10 min)

```
python inventar.py tinte.json --prag 1000
python inventar.py tinte.json             # pragul implicit, 1024
git add raport_inventar.json
git commit -m "Raportul uneltei reparate"
git push
git log --oneline
```

În git log --oneline trebuie să vezi, de jos în sus: pornirea, cele zece reparații (fiecare cu mesajul ei) și raportul. Același istoric îl vezi pe GitHub.

Predai: linkul depozitului exercitiu-git, în care depanare/ conține `inventar.py` reparat și `raport_inventar.json`, iar istoria are câte un commit pentru fiecare reparație.

OPȚIONAL (pentru vârful sălii sau acasă). recon.py citit și extins

Dacă ai terminat ambele lucrări: deschide propria unealtă arsenal/recon.py și extinde-o. Lucrezi în depozitul tău de curs, cu commit după fiecare pas.

1. Fișa de citire (arsenal/citire_recon.md): ce tip întoarce adrese()? ce se întâmplă când un tip DNS nu are răspuns? rescrie [x.to_text() for x in r] ca buclă; ce ar afișa unealta fără return rez?

2. O funcție rezumat(fisa) care ÎNTOARCE un dict tip DNS -> număr de înregistrări.

3. Opțiunea --tipuri A,MX: textul despărțit cu .split(","), iar adrese(domeniu, tipuri=...) are o valoare implicită.

4. Opțiunea --din fisa.json: citește o fișă salvată, fără rețea; dacă fișierul lipsește, spune clar, nu crapă (try/except FileNotFoundError).

5. Un test pentru rezumat() cu o fișă scrisă de mână, care trece fără internet.

Ce predai și cum se notează

Livrabil	Ce conține
`exercitiu-git/depanare/`	inventar.py reparat și raport_inventar.json (B1-B6).
istoria din exercitiu-git	un commit pentru fiecare dintre cele zece reparații, cu mesaj clar (B2-B5).

La verificare, asistentul deschide linkurile, citește istoria și îți poate cere să explici oricare commit sau oricare reparație. Istoria se scrie cinstit: predai ce ți-a ieșit, cu sursele notate dacă ai cerut ajutor.